

```
var car = clone (vehicle);
car.tyres = 4;
```

In this case, the prototype for the car object is *vehicle*, and car inherits the wheels property from *vehicle*. The next statement extends the properties of the *car* by adding a property for *tyres*. Note that this is different from languages like C++ or Java, where the members of an object are fixed, based on the class definition (except through inheritance) and cannot be extended at will. The point to be noted here is that properties inherited by *car* from its prototype are not stored in the *car* object; when such a property is referenced, if it is not found in *car*, then its prototype (*vehicle*) is searched. Recall that each object has a *prototype* field. When a property is looked up, it starts by searching the current object, then its parent, followed by its parent's parent, until that property is found. Note that this kind of recursive search up the prototype chain can impose considerable overhead in JavaScript implementations. While there are a number of other interesting language features in JavaScript, I will not discuss them further. Instead, interested readers can find the details in any JavaScript book such as 'JavaScript—The Definitive Guide' from O'Reilly Media, or 'HeadFirst JavaScript'. Next, let us explore JavaScript engines.

JavaScript engines

JavaScript is a high-level language and an engine is needed to run the code. A JavaScript engine typically contains a parser, an interpreter and a runtime module. The parser parses the JavaScript code into tokens, typically building an abstract syntax tree to represent the JavaScript program. The interpreter walks the abstract syntax tree and interprets the intermediate representation contained in it. While earlier versions of JavaScript engines used the simpler AST walking interpreter design, things have changed quite a bit in later engines. These parse the JavaScript to an intermediate form, which typically contains byte-codes. An interpreter translates the byte-codes and optionally supports a JIT compiler, which can translate byte-codes to native code. There are trade-offs involved in having an interpreter rather than a JavaScript compiler. Interpreters are faster on application start-up, because they don't spend considerable overhead in creating optimised/efficient internal representations; hence, their start-up latency is quite low. On the other hand, a JavaScript compiler can convert the intermediate representation into machine or native code—and, of course, native code execution is considerably faster than interpreted code.

The very first JavaScript engine, created by Brendan Eich, who invented JavaScript, was code-named SpiderMonkey and was implemented in C. Rhino, from Mozilla Corporation (<http://www.mozilla.org/rhino/>), is an open source implementation of a JavaScript engine written in Java. The SquirrelFish Extreme engine (code named Nitro in the Safari browser) uses interpreted byte-codes with certain hot parts of the code being translated by the JIT compiler into machine language. TraceMonkey, embedded in Mozilla's Firefox browser, employs a similar technology. The internals

of TraceMonkey are described in a paper presented at the ACM Conference on Programming Languages Design and Implementation (PLDI) 2009, which is available at <http://www.cs.ucsb.edu/~ckrintz/gal-pldi09.pdf>.

Another popular JavaScript engine is Google's V8, which is used in the Chrome browser. Details of V8 are available at <https://developers.google.com/v8/>. V8 compiles JavaScript into native x86 code, which makes it quite fast. It also employs a number of optimisations. Recall that I had earlier mentioned the overheads in recursive property lookup following prototype chains. V8 tries to avoid this overhead by supporting fast property access, by what are known as hidden classes. These are based on the premise that an object's properties are typically unlikely to change after it has been initialised. Hence the object structure is captured in hidden classes, and an object's hidden class changes when a new property is added. The concept of hidden classes is not new, but inherited from the first experimental prototype language called 'Self' (www.selflanguage.org/). More details on V8's hidden classes can be found at <https://developers.google.com/v8/design>. I will discuss V8 internals in detail in the next column.

My 'must-read book' for this month

This month's 'must-read book' suggestion comes from one of our readers, Suresh C, who writes, "I am basically a Windows programmer (Win32 and kernel), but once I started reading LFY and the articles in it, I started exploring Linux user-mode programming (using glibc calls) and a bit of kernel programming. The articles in LFY are very good starting points. So here is a book, which I feel is a must-read for any person who is willing to master Linux concepts and programming—"Linux Programming Interface" by Michael Kerrisk." Suresh recommends the book for the following reasons:

- In-depth coverage of Linux OS concepts.
- Examples in each section to make the concepts clear.
- Explanations that are easy to understand.

If you have a favourite programming book/article that you think is a must-read for every programmer, please do send me a note with the book's name and a short write-up on why you think it is useful, so that I can feature it in the column. This would help many readers who want to improve their coding skills.

If you have any favourite programming puzzles that you would like to discuss on this forum, please send them to me, along with your solutions and feedback, at sandyasm_AT_yahoo_DOT_com. Till we meet again next month, happy programming and here's wishing you the very best! 

By: Sandya Mannarswamy

The author is an expert in systems software and works at Hewlett-Packard India. Her interests include compilers, multi-core technologies and software development tools. If you are preparing for systems software interviews, you may find it useful to visit Sandya's LinkedIn group: 'Computer Science Interview Training India' at <http://www.linkedin.com/groups?home=&gid=2339182>.